ANSI SQL Using MySQL

Table Creation:

```
1) CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    city VARCHAR(100) NOT NULL,
    registration_date DATE NOT NULL);

2) CREATE TABLE Events (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    description TEXT,
    city VARCHAR(100) NOT NULL,
    start_date DATETIME NOT NULL,
    end_date DATETIME NOT NULL,
    status ENUM('upcoming','completed','cancelled'),
    organizer_id INT,
    FOREIGN KEY (organizer_id) REFERENCES Users(user_id));

3) CREATE TABLE Sessions (
    session_id INT AUTO_INCREMENT PRIMARY KEY,
    event_id INT,
    title VARCHAR(200) NOT NULL,
    speaker_name VARCHAR(100) NOT NULL,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    FOREIGN KEY (event_id) REFERENCES Events(event_id));
```

4) CREATE TABLE Registrations (

registration_id INT AUTO_INCREMENT PRIMARY KEY,

user_id INT,

event_id INT,

registration_date DATE NOT NULL,

FOREIGN KEY (user_id) REFERENCES Users(user_id),

FOREIGN KEY (event_id) REFERENCES Events(event_id));

5) CREATE TABLE Feedback (

feedback_id INT AUTO_INCREMENT PRIMARY KEY,

user_id INT,

event_id INT,

rating INT CHECK (rating BETWEEN 1 AND 5),

comments TEXT,

feedback_date DATE NOT NULL,

FOREIGN KEY (user_id) REFERENCES Users(user_id),

FOREIGN KEY (event_id) REFERENCES Events(event_id));

6) CREATE TABLE Resources (

resource_id INT AUTO_INCREMENT PRIMARY KEY,

event_id INT,

resource_type ENUM('pdf','image','link'),

resource_url VARCHAR(255) NOT NULL,

uploaded_at DATETIME NOT NULL,

FOREIGN KEY (event_id) REFERENCES Events(event_id));

Values Insertion:

1) INSERT INTO Users (user_id, full_name, email, city, registration_date) VALUES

(1,'Alice Johnson','alice@example.com','New York','2024-12-01'),

(2,'Bob Smith','bob@example.com','Los Angeles','2024-12-05'),

(3,'Charlie Lee','charlie@example.com','Chicago','2024-12-10'),

(4,'Diana King','diana@example.com','New York','2025-01-15'),

(5,'Ethan Hunt','ethan@example.com','Los Angeles','2025-02-01');

2) INSERT INTO Events (event_id, title, description, city, start_date, end_date, status, organizer_id) VALUES

(1,'Tech Innovators Meetup','A meetup for tech enthusiasts.','New York','2025-06-10 10:00:00','2025-06-10 16:00:00','upcoming',1),

(2,'AI & ML Conference','Conference on AI and ML advancements.','Chicago','2025-05-15 09:00:00','2025-05-15 17:00:00','completed',3),

(3,'Frontend Development Bootcamp','Hands-on training on frontend tech.','Los Angeles','2025-07-01 10:00:00','2025-07-03 16:00:00','upcoming',2);

3) INSERT INTO Sessions (session_id, event_id, title, speaker_name, start_time, end_time) VALUES

(1,1,'Opening Keynote','Dr. Tech','2025-06-10 10:00:00','2025-06-10 11:00:00'),

(2,1,'Future of Web Dev','Alice Johnson','2025-06-10 11:15:00','2025-06-10 12:30:00'),

(3,2,'AI in Healthcare','Charlie Lee','2025-05-15 09:30:00','2025-05-15 11:00:00'),

(4,3,'Intro to HTML5','Bob Smith','2025-07-01 10:00:00','2025-07-01 12:00:00');

4) INSERT INTO Registrations (registration_id, user_id, event_id, registration_date) VALUES

(1,1,1,'2025-05-01'),

(2,2,1,'2025-05-02'),

(3,3,2,'2025-04-30'),

(4,4,2,'2025-04-28'),

(5,5,3,'2025-06-15');

5) INSERT INTO Feedback (feedback_id, user_id, event_id, rating, comments, feedback_date) VALUES

(1,3,2,4,'Great insights!','2025-05-16'),

(2,4,2,5,'Very informative.','2025-05-16'),

(3,2,1,3,'Could be better.','2025-06-11');

6) INSERT INTO Resources (resource_id, event_id, resource_type, resource_url, uploaded_at) VALUES

(1,1,'pdf','https://portal.com/resources/tech_meetup_agenda.pdf','2025-05-01 10:00:00'),

(2,2,'image','https://portal.com/resources/ai_poster.jpg','2025-04-20 09:00:00'),

(3,3,'link','https://portal.com/resources/html5_docs','2025-06-25 15:00:00');

Exercises:

1) SELECT u.full_name,e.title,e.city,e.start_date

FROM Users u

JOIN Registrations r ON u.user_id=r.user_id

JOIN Events e ON r.event_id=e.event_id

WHERE e.status='upcoming'

AND u.city=e.city

ORDER BY e.start_date;

2) SELECT e.event_id,e.title,AVG(f.rating) AS avg_rating

FROM Events e

JOIN Feedback f ON e.event_id=f.event_id

GROUP BY e.event_id,e.title

HAVING COUNT(f.feedback_id)>=10

ORDER BY avg_rating DESC;

3) SELECT u.user_id,u.full_name

FROM Users u

LEFT JOIN Registrations r ON u.user_id=r.user_id

GROUP BY u.user_id,u.full_name

HAVING MAX(r.registration_date) IS NULL

OR MAX(r.registration_date)<CURDATE()-INTERVAL 90 DAY;

4) SELECT e.event_id,e.title,COUNT(s.session_id) AS session_count

FROM Events e

LEFT JOIN Sessions s ON e.event_id=s.event_id

WHERE TIME(s.start_time)>='10:00:00'

AND TIME(s.start_time)<'12:00:00'

GROUP BY e.event_id,e.title;

5) SELECT e.city,COUNT(DISTINCT r.user_id) AS registrations

FROM Events e

JOIN Registrations r ON e.event_id=r.event_id

GROUP BY e.city

ORDER BY registrations DESC

LIMIT 5;

6) SELECT e.event_id,e.title,

COUNT(CASE WHEN r.resource_type='pdf' THEN 1 END) AS pdf_count,

COUNT(CASE WHEN r.resource_type='image' THEN 1 END) AS image_count,

COUNT(CASE WHEN r.resource_type='link' THEN 1 END) AS link_count

FROM Events e

LEFT JOIN Resources r ON e.event_id=r.event_id

GROUP BY e.event_id,e.title;

7) SELECT u.full_name,e.title,f.comments,f.rating

FROM Feedback f

JOIN Users u ON f.user_id=u.user_id

JOIN Events e ON f.event_id=e.event_id

WHERE f.rating<3;

8) SELECT e.event_id,e.title,COUNT(s.session_id) AS total_sessions

FROM Events e

LEFT JOIN Sessions s ON e.event_id=s.event_id

WHERE e.status='upcoming'

GROUP BY e.event_id,e.title;

9) SELECT u.full_name,e.status,COUNT(e.event_id) AS total_events

FROM Users u

JOIN Events e ON u.user_id=e.organizer_id

GROUP BY u.full_name,e.status;

10) SELECT e.event_id,e.title

FROM Events e

JOIN Registrations r ON e.event_id=r.event_id

LEFT JOIN Feedback f ON e.event_id=f.event_id

WHERE f.feedback_id IS NULL

GROUP BY e.event_id,e.title;

11) SELECT registration_date,COUNT(*) AS user_count

FROM Users

WHERE registration_date>=CURDATE()-INTERVAL 7 DAY

GROUP BY registration_date;

12) SELECT e.event_id,e.title,COUNT(s.session_id) AS total_sessions

FROM Events e

JOIN Sessions s ON e.event_id=s.event_id

GROUP BY e.event_id,e.title

HAVING COUNT(s.session_id)=(

SELECT MAX(session_total)

FROM(

SELECT COUNT(*) AS session_total

FROM Sessions

GROUP BY event_id) x);

13) SELECT e.city,AVG(f.rating) AS avg_rating

FROM Events e

JOIN Feedback f ON e.event_id=f.event_id

GROUP BY e.city;

14) SELECT e.event_id,e.title,COUNT(r.registration_id) AS registrations

FROM Events e

JOIN Registrations r ON e.event_id=r.event_id

GROUP BY e.event_id,e.title

ORDER BY registrations DESC

LIMIT 3;

15) SELECT s1.event_id,

s1.title AS session1,

s2.title AS session2

FROM Sessions s1

JOIN Sessions s2

ON s1.event_id=s2.event_id

```
AND s1.session_id<s2.session_id
AND s1.start_time<s2.end_time
AND s1.end_time>s2.start_time;

16) SELECT u.user_id,u.full_name
FROM Users u
LEFT JOIN Registrations r ON u.user_id=r.user_id
WHERE u.registration_date>=CURDATE()-INTERVAL 30 DAY
AND r.registration_id IS NULL;

17) SELECT speaker_name,COUNT(*) AS total_sessions
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(*)>1;

18) SELECT e.event_id,e.title
FROM Events e
LEFT JOIN Resources r ON e.event_id=r.event_id
WHERE r.resource_id IS NULL;

19) SELECT e.event_id,e.title,
COUNT(DISTINCT r.registration_id) AS total_registrations,
AVG(f.rating) AS avg_rating
FROM Events e
LEFT JOIN Registrations r ON e.event_id=r.event_id
LEFT JOIN Feedback f ON e.event_id=f.event_id
WHERE e.status='completed'
GROUP BY e.event_id,e.title;
```

20) SELECT u.user_id,u.full_name,

COUNT(DISTINCT r.event_id) AS events_attended,

COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted

FROM Users u

LEFT JOIN Registrations r ON u.user_id=r.user_id

LEFT JOIN Feedback f ON u.user_id=f.user_id

GROUP BY u.user_id,u.full_name;

21) SELECT u.user_id,u.full_name,COUNT(f.feedback_id) AS feedback_count

FROM Users u

JOIN Feedback f ON u.user_id=f.user_id

GROUP BY u.user_id,u.full_name

ORDER BY feedback_count DESC

LIMIT 5;

22) SELECT user_id,event_id,COUNT(*) AS duplicate_count

FROM Registrations

GROUP BY user_id,event_id

HAVING COUNT(*)>1;

23) SELECT YEAR(registration_date) AS year,MONTH(registration_date) AS month,

COUNT(*) AS registrations FROM Registrations WHERE registration_date>=CURDATE()-INTERVAL 12 MONTH

GROUP BY YEAR(registration_date), MONTH(registration_date) ORDER BY year,month;

24) SELECT e.event_id,e.title,

AVG(TIMESTAMPDIFF(MINUTE,s.start_time,s.end_time)) AS avg_duration

FROM Events e JOIN Sessions s ON e.event_id=s.event_id

GROUP BY e.event_id,e.title;